

Chương 13 – Tải và Tiền xử lý Dữ liệu với TensorFlow

Notebook này chứa tất cả các mã nguồn mẫu và lời giải cho các bài tập trong chương 13.

 [Open in Colab](#)  [Open in Kaggle](#)

▼ Cài đặt (Setup)

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Và TensorFlow ≥ 2.8 :

```
import tensorflow as tf

assert version.parse(tf.__version__) >= version.parse("2.8.0")
```

▼ API tf.data

```
import tensorflow as tf

X = tf.range(10) # any data tensor
dataset = tf.data.Dataset.from_tensor_slices(X)
dataset

<TensorSliceDataset shapes: (), types: tf.int32>
```

```
for item in dataset:
    print(item)
```

```
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
tf.Tensor(2, shape=(), dtype=int32)
tf.Tensor(3, shape=(), dtype=int32)
tf.Tensor(4, shape=(), dtype=int32)
tf.Tensor(5, shape=(), dtype=int32)
tf.Tensor(6, shape=(), dtype=int32)
tf.Tensor(7, shape=(), dtype=int32)
tf.Tensor(8, shape=(), dtype=int32)
tf.Tensor(9, shape=(), dtype=int32)
```

```
X_nested = {"a": ([1, 2, 3], [4, 5, 6]), "b": [7, 8, 9]}
dataset = tf.data.Dataset.from_tensor_slices(X_nested)
for item in dataset:
    print(item)
```

```
{'a': (<tf.Tensor: shape=(), dtype=int32, numpy=1>, <tf.Tensor: shape=(), dtype=int32, numpy=4>), 'b': <tf.Tensor: shape=(), dtype=int32, numpy=7>},
{'a': (<tf.Tensor: shape=(), dtype=int32, numpy=2>, <tf.Tensor: shape=(), dtype=int32, numpy=5>), 'b': <tf.Tensor: shape=(), dtype=int32, numpy=8>},
{'a': (<tf.Tensor: shape=(), dtype=int32, numpy=3>, <tf.Tensor: shape=(), dtype=int32, numpy=6>), 'b': <tf.Tensor: shape=(), dtype=int32, numpy=9>}
```

▼ Chaining Transformations (Chuỗi các phép biến đổi)

```
dataset = tf.data.Dataset.from_tensor_slices(tf.range(10))
dataset = dataset.repeat(3).batch(7)
for item in dataset:
    print(item)
```

```
tf.Tensor([0 1 2 3 4 5 6], shape=(7,), dtype=int32)
tf.Tensor([7 8 9 0 1 2 3], shape=(7,), dtype=int32)
tf.Tensor([4 5 6 7 8 9 0], shape=(7,), dtype=int32)
tf.Tensor([1 2 3 4 5 6 7], shape=(7,), dtype=int32)
tf.Tensor([8 9], shape=(2,), dtype=int32)
```

```
dataset = dataset.map(lambda x: x * 2) # x is a batch
for item in dataset:
    print(item)
```

```
tf.Tensor([ 0  2  4  6  8 10 12], shape=(7,), dtype=int32)
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)
tf.Tensor([ 2  4  6  8 10 12 14], shape=(7,), dtype=int32)
tf.Tensor([16 18], shape=(2,), dtype=int32)
```

```
dataset = dataset.filter(lambda x: tf.reduce_sum(x) > 50)
for item in dataset:
    print(item)
```

```
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)
tf.Tensor([ 2  4  6  8 10 12 14], shape=(7,), dtype=int32)
```

```
for item in dataset.take(2):
    print(item)
```

```
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)
```

✓ Shuffling the Data (Trộn dữ liệu)

```
dataset = tf.data.Dataset.range(10).repeat(2)
dataset = dataset.shuffle(buffer_size=4, seed=42).batch(7)
for item in dataset:
    print(item)
```

```
tf.Tensor([1 4 2 3 5 0 6], shape=(7,), dtype=int64)
tf.Tensor([9 8 2 0 3 1 4], shape=(7,), dtype=int64)
tf.Tensor([5 7 9 6 7 8], shape=(6,), dtype=int64)
```

✓ Interleaving lines from multiple files (Đan xen các dòng từ nhiều tệp)

Let's start by loading and preparing the California housing dataset. We first load it, then split it into a training set, a validation set and a test set:

```
# extra code - fetches, splits and normalizes the California housing dataset

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target.reshape(-1, 1), random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)
```

For a very large dataset that does not fit in memory, you will typically want to split it into many files first, then have TensorFlow read these files in parallel. To demonstrate this, let's start by splitting the housing dataset and saving it to 20 CSV files:

```
# extra code - split the dataset into 20 parts and save it to CSV files

import numpy as np
from pathlib import Path
```

```
def save_to_csv_files(data, name_prefix, header=None, n_parts=10):
    housing_dir = Path() / "datasets" / "housing"
    housing_dir.mkdir(parents=True, exist_ok=True)
    filename_format = "my_{:02d}.csv"

    filepaths = []
    m = len(data)
    chunks = np.array_split(np.arange(m), n_parts)
    for file_idx, row_indices in enumerate(chunks):
        part_csv = housing_dir / filename_format.format(name_prefix, file_idx)
        filepaths.append(str(part_csv))
        with open(part_csv, "w") as f:
            if header is not None:
                f.write(header)
                f.write("\n")
            for row_idx in row_indices:
                f.write(",".join([str(col) for col in data[row_idx]]))
                f.write("\n")
    return filepaths

train_data = np.c_[X_train, y_train]
valid_data = np.c_[X_valid, y_valid]
test_data = np.c_[X_test, y_test]
header_cols = housing.feature_names + ["MedianHouseValue"]
header = ",".join(header_cols)

train_filepaths = save_to_csv_files(train_data, "train", header, n_parts=20)
valid_filepaths = save_to_csv_files(valid_data, "valid", header, n_parts=10)
test_filepaths = save_to_csv_files(test_data, "test", header, n_parts=10)
```

Okay, now let's take a peek at the first few lines of one of these CSV files:

```
print(",".join(open(train_filepaths[0]).readlines()[:4]))
```

```
MedInc,HouseAge,AveRooms,AveBedrms,Population,AveOccup,Longitude,MedianHouseValue
3.5214,15.0,3.0499445061043287,1.106548279689234,1447.0,1.6059933407325193,37.63,-122.43,1.442
5.3275,5.0,6.490059642147117,0.9910536779324056,3464.0,3.4433399602385686,33.69,-117.39,1.687
3.1,29.0,7.5423728813559325,1.5915254237288134,1328.0,2.2508474576271187,38.44,-122.98,1.621
```

```
train_filepaths
```

```
['datasets/housing/my_train_00.csv',
'datasets/housing/my_train_01.csv',
'datasets/housing/my_train_02.csv',
'datasets/housing/my_train_03.csv',
'datasets/housing/my_train_04.csv',
'datasets/housing/my_train_05.csv',
'datasets/housing/my_train_06.csv',
'datasets/housing/my_train_07.csv',
'datasets/housing/my_train_08.csv',
'datasets/housing/my_train_09.csv',
'datasets/housing/my_train_10.csv',
'datasets/housing/my_train_11.csv',
'datasets/housing/my_train_12.csv',
'datasets/housing/my_train_13.csv',
'datasets/housing/my_train_14.csv',
'datasets/housing/my_train_15.csv',
'datasets/housing/my_train_16.csv',
'datasets/housing/my_train_17.csv',
'datasets/housing/my_train_18.csv',
'datasets/housing/my_train_19.csv']
```

Building an Input Pipeline

```
filepath_dataset = tf.data.Dataset.list_files(train_filepaths, seed=42)
```

```
# extra code - shows that the file paths are shuffled
for filepath in filepath_dataset:
    print(filepath)
```

```
tf.Tensor(b'datasets/housing/my_train_05.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_16.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_01.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_17.csv', shape=(), dtype=string)
```

```
tf.Tensor(b'datasets/housing/my_train_00.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_14.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_10.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_02.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_12.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_19.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_07.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_09.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_13.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_15.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_11.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_18.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_04.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_06.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_03.csv', shape=(), dtype=string)
tf.Tensor(b'datasets/housing/my_train_08.csv', shape=(), dtype=string)
```

```
n_readers = 5
dataset = filepath_dataset.interleave(
    lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
    cycle_length=n_readers)
```

```
for line in dataset.take(5):
    print(line)
```

```
tf.Tensor(b'4.5909,16.0,5.475877192982456,1.0964912280701755,1357.0,2.9758771929824563,33.63,-117.71,2.418', shape=(), dtype=string)
tf.Tensor(b'2.4792,24.0,3.4547038327526134,1.1341463414634145,2251.0,3.921602787456446,34.18,-118.38,2.0', shape=(), dtype=string)
tf.Tensor(b'4.2708,45.0,5.121387283236994,0.953757225433526,492.0,2.8439306358381504,37.48,-122.19,2.67', shape=(), dtype=string)
tf.Tensor(b'2.1856,41.0,3.7189873417721517,1.0658227848101265,803.0,2.0329113924050635,32.76,-117.12,1.205', shape=(), dtype=string)
tf.Tensor(b'4.1812,52.0,5.701388888888889,0.9965277777777778,692.0,2.4027777777777777,33.73,-118.31,3.215', shape=(), dtype=string)
```

Preprocessing the Data

```
# extra code - compute the mean and standard deviation of each feature
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
scaler.fit(X_train)
```

```
StandardScaler()
```

```
X_mean, X_std = scaler.mean_, scaler.scale_ # extra code
n_inputs = 8
```

```
def parse_csv_line(line):
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    fields = tf.io.decode_csv(line, record_defaults=defs)
    return tf.stack(fields[:-1]), tf.stack(fields[-1:])
```

```
def preprocess(line):
    x, y = parse_csv_line(line)
    return (x - X_mean) / X_std, y
```

```
preprocess(b'4.2083,44.0,5.3232,0.9171,846.0,2.3370,37.47,-122.2,2.782')
```

```
(<tf.Tensor: shape=(8,), dtype=float32, numpy=
array([ 0.16579159,  1.216324 , -0.05204564, -0.39215982, -0.5277444 ,
        -0.2633488 ,  0.8543046 , -1.3072058 ], dtype=float32)>,
 <tf.Tensor: shape=(1,), dtype=float32, numpy=array([2.782], dtype=float32)>)
```

Putting Everything Together + Prefetching

```
def csv_reader_dataset(filepaths, n_readers=5, n_read_threads=None,
                       n_parse_threads=5, shuffle_buffer_size=10_000, seed=42,
                       batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths, seed=seed)
    dataset = dataset.interleave(
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers, num_parallel_calls=n_read_threads)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
```

```
dataset = dataset.shuffle(shuffle_buffer_size, seed=seed)
return dataset.batch(batch_size).prefetch(1)
```

extra code - show the first couple of batches produced by the dataset

```
example_set = csv_reader_dataset(train_filepaths, batch_size=3)
for X_batch, y_batch in example_set.take(2):
    print("X =", X_batch)
    print("y =", y_batch)
    print()
```

```
X = tf.Tensor(
[[[-1.3957452 -0.04940685 -0.22830808  0.22648273  2.2593622  0.35200632
   0.9667386 -1.4121602 ]
 [ 2.7112627 -1.0778131  0.69413143 -0.14870553  0.51810503  0.3507294
 -0.82285154  0.80680597]
 [-0.13484643 -1.868895  0.01032507 -0.13787179 -0.12893449  0.03143518
  0.2687057  0.13212144]], shape=(3, 8), dtype=float32)
```

```
y = tf.Tensor(
[[1.819]
 [3.674]
 [0.954]], shape=(3, 1), dtype=float32)
```

```
X = tf.Tensor(
[[ 0.09031774  0.9789995  0.1327582 -0.13753782 -0.23388447  0.10211545
   0.97610843 -1.4121602 ]
 [ 0.05218809 -2.0271113  0.2940109 -0.02403445  0.16218767 -0.02844518
  1.4117942 -0.93737936]
 [-0.672276  0.02970133 -0.76922584 -0.15086786  0.4962024 -0.02741998
 -0.7853724  0.77182245]], shape=(3, 8), dtype=float32)
```

```
y = tf.Tensor(
[[2.725]
 [1.205]
 [1.625]], shape=(3, 1), dtype=float32)
```

Here is a short description of each method in the `Dataset` class:

```
# extra code - list all methods of the tf.data.Dataset class
for m in dir(tf.data.Dataset):
    if not (m.startswith("_") or m.endswith("_")):
        func = getattr(tf.data.Dataset, m)
        if hasattr(func, "__doc__"):
            print("• {21s}{3}".format(m + "()", func.__doc__.split("\n")[0]))
```

- `apply()` Applies a transformation function to this dataset.
- `as_numpy_iterator()` Returns an iterator which converts all elements of the dataset to numpy.
- `batch()` Combines consecutive elements of this dataset into batches.
- `bucket_by_sequence_length()` A transformation that buckets elements in a `Dataset` by length.
- `cache()` Caches the elements in this dataset.
- `cardinality()` Returns the cardinality of the dataset, if known.
- `choose_from_datasets()` Creates a dataset that deterministically chooses elements from `datasets`.
- `concatenate()` Creates a `Dataset` by concatenating the given dataset with this dataset.
- `element_spec()` The type specification of an element of this dataset.
- `enumerate()` Enumerates the elements of this dataset.
- `filter()` Filters this dataset according to `predicate`.
- `flat_map()` Maps `map\_func` across this dataset and flattens the result.
- `from_generator()` Creates a `Dataset` whose elements are generated by `generator`. (deprecated arguments)
- `from_tensor_slices()` Creates a `Dataset` whose elements are slices of the given tensors.
- `from_tensors()` Creates a `Dataset` with a single element, comprising the given tensors.
- `get_single_element()` Returns the single element of the `dataset`.
- `group_by_window()` Groups windows of elements by key and reduces them.
- `interleave()` Maps `map\_func` across this dataset, and interleaves the results.
- `list_files()` A dataset of all files matching one or more glob patterns.
- `map()` Maps `map\_func` across the elements of this dataset.
- `options()` Returns the options for this dataset and its inputs.
- `padded_batch()` Combines consecutive elements of this dataset into padded batches.
- `prefetch()` Creates a `Dataset` that prefetches elements from this dataset.
- `random()` Creates a `Dataset` of pseudorandom values.
- `range()` Creates a `Dataset` of a step-separated range of values.
- `reduce()` Reduces the input dataset to a single element.
- `rejection_resample()` A transformation that resamples a dataset to a target distribution.
- `repeat()` Repeats this dataset so each original value is seen `count` times.
- `sample_from_datasets()` Samples elements at random from the datasets in `datasets`.
- `scan()` A transformation that scans a function across an input dataset.
- `shard()` Creates a `Dataset` that includes only 1/`num\_shards` of this dataset.
- `shuffle()` Randomly shuffles the elements of this dataset.
- `skip()` Creates a `Dataset` that skips `count` elements from this dataset.
- `snapshot()` API to persist the output of the input dataset.
- `take()` Creates a `Dataset` with at most `count` elements from this dataset.

- `take_while()` A transformation that stops dataset iteration based on a ``predicate``.
- `unbatch()` Splits elements of a dataset into multiple elements.
- `unique()` A transformation that discards duplicate elements of a ``Dataset``.
- `window()` Returns a dataset of "windows".
- `with_options()` Returns a new ``tf.data.Dataset`` with the given options set.
- `zip()` Creates a ``Dataset`` by zipping together the given datasets.

▼ Using the Dataset with Keras

```
train_set = csv_reader_dataset(train_filepaths)
valid_set = csv_reader_dataset(valid_filepaths)
test_set = csv_reader_dataset(test_filepaths)
```

```
# extra code - for reproducibility
tf.keras.backend.clear_session()
tf.random.set_seed(42)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(30, activation="relu", kernel_initializer="he_normal",
                           input_shape=X_train.shape[1:]),
    tf.keras.layers.Dense(1),
])
model.compile(loss="mse", optimizer="sgd")
model.fit(train_set, validation_data=valid_set, epochs=5)
```

```
Epoch 1/5
363/363 [=====] - 1s 2ms/step - loss: 1.3569 - val_loss: 0.5272
Epoch 2/5
363/363 [=====] - 0s 965us/step - loss: 0.5132 - val_loss: 63.7862
Epoch 3/5
363/363 [=====] - 0s 902us/step - loss: 0.5916 - val_loss: 20.3634
Epoch 4/5
363/363 [=====] - 1s 944us/step - loss: 0.5089 - val_loss: 0.3993
Epoch 5/5
363/363 [=====] - 1s 905us/step - loss: 0.4200 - val_loss: 0.3639
<keras.callbacks.History at 0x7f82912a32b0>
```

```
test_mse = model.evaluate(test_set)
new_set = test_set.take(3) # pretend we have 3 new samples
y_pred = model.predict(new_set) # or you could just pass a NumPy array
```

```
162/162 [=====] - 0s 594us/step - loss: 0.3868
```

Warning: the `mean_squared_error()` function no longer exists. You must now use an instance of the `MeanSquaredError` class instead.

```
# extra code - defines the optimizer and loss function for training
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.MeanSquaredError()

n_epochs = 5
for epoch in range(n_epochs):
    for X_batch, y_batch in train_set:
        # extra code - perform one Gradient Descent step
        # as explained in Chapter 12
        print("\rEpoch {}/{}".format(epoch + 1, n_epochs), end="")
        with tf.GradientTape() as tape:
            y_pred = model(X_batch)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)
            gradients = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```

```
Epoch 5/5
```

```
@tf.function
def train_one_epoch(model, optimizer, loss_fn, train_set):
    for X_batch, y_batch in train_set:
        with tf.GradientTape() as tape:
            y_pred = model(X_batch)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)
            gradients = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```



```
# extra code - shows that the data is decompressed correctly
for item in dataset:
    print(item)

tf.Tensor(b'Compress, compress, compress!', shape=(), dtype=string)
```

▼ A Brief Introduction to Protocol Buffers

For this section you need to [install protobuf](#). In general you will not have to do so when using TensorFlow, as it comes with functions to create and parse protocol buffers of type `tf.train.Example`, which are generally sufficient. However, in this section we will learn about protocol buffers by creating our own simple protobuf definition, so we need the protobuf compiler (`protoc`): we will use it to compile the protobuf definition to a Python module that we can then use in our code.

First let's write a simple protobuf definition:

```
%%writefile person.proto
syntax = "proto3";
message Person {
    string name = 1;
    int32 id = 2;
    repeated string email = 3;
}
```

Overwriting person.proto

Warning: Colab currently has an old version of the protobuf compiler installed. We need to uninstall it and manually install a more recent version.

```
import os

IS_COLAB = "google.colab" in sys.modules

if IS_COLAB:
    !apt remove -y protobuf-compiler
    PB_REL="https://github.com/protocolbuffers/protobuf/releases"
    !curl -LO {PB_REL}/download/v25.1/protoc-25.1-linux-x86_64.zip
    !unzip protoc-25.1-linux-x86_64.zip -d /root/.local
    os.environ["PATH"] += ":/root/.local/bin"
```

And let's compile it (the `--descriptor_set_out` and `--include_imports` options are only required for the `tf.io.decode_proto()` example below):

```
!protoc person.proto --python_out=. --descriptor_set_out=person.desc --include_imports
```

```
%ls person*
```

```
person.desc  person.proto  person_pb2.py
```

```
from person_pb2 import Person # import the generated access class

person = Person(name="Al", id=123, email=["a@b.com"]) # create a Person
print(person) # display the Person
```

```
name: "Al"
id: 123
email: "a@b.com"
```

```
person.name # read a field
```

```
'Al'
```

```
person.name = "Alice" # modify a field
```

```
person.email[0] # repeated fields can be accessed like arrays
```



```
'a@b.com'
```

```
person.email.append("c@d.com") # add an email address
```

```
serialized = person.SerializeToString() # serialize person to a byte string  
serialized
```

```
b'\n\x05Alice\x10{\x1a\x07a@b.com\x1a\x07c@d.com'
```

```
person2 = Person() # create a new Person  
person2.ParseFromString(serialized) # parse the byte string (27 bytes long)
```

```
27
```

```
person == person2 # now they are equal
```

```
True
```

Custom protobuf

In rare cases, you may want to parse a custom protobuf (like the one we just created) in TensorFlow. For this you can use the `tf.io.decode_proto()` function:

```
# extra code - shows how to use the tf.io.decode_proto() function
```

```
person_tf = tf.io.decode_proto(  
    bytes=serialized,  
    message_type="Person",  
    field_names=["name", "id", "email"],  
    output_types=[tf.string, tf.int32, tf.string],  
    descriptor_source="person.desc")
```

```
person_tf.values
```

```
[<tf.Tensor: shape=(1,), dtype=string, numpy=array([b'Alice'], dtype=object)>,  
<tf.Tensor: shape=(1,), dtype=int32, numpy=array([123], dtype=int32)>,  
<tf.Tensor: shape=(2,), dtype=string, numpy=array([b'a@b.com', b'c@d.com'], dtype=object)>]
```

For more details, see the [tf.io.decode_proto\(\)](#) documentation.

TensorFlow Protobufs

Here is the definition of the `tf.train.Example` protobuf:

```
syntax = "proto3";  
  
message BytesList { repeated bytes value = 1; }  
message FloatList { repeated float value = 1 [packed = true]; }  
message Int64List { repeated int64 value = 1 [packed = true]; }  
message Feature {  
    oneof kind {  
        BytesList bytes_list = 1;  
        FloatList float_list = 2;  
        Int64List int64_list = 3;  
    }  
};  
  
message Features { map<string, Feature> feature = 1; };  
message Example { Features features = 1; };
```

```
from tensorflow.train import BytesList, FloatList, Int64List  
from tensorflow.train import Feature, Features, Example  
  
person_example = Example(  
    features=Features(  
        name=Feature(  
            bytes_list=BytesList([b'Alice'])  
        ),  
        id=Feature(  
            int64_list=Int64List([123])  
        ),  
        email=Feature(  
            bytes_list=BytesList([b'a@b.com', b'c@d.com'])  
        )  
    )  
)
```

```

features=Features(
    feature={
        "name": Feature(bytes_list=BytesList(value=[b"Alice"])),
        "id": Feature(int64_list=Int64List(value=[123])),
        "emails": Feature(bytes_list=BytesList(value=[b"a@b.com",
                                                    b"c@d.com"])))
    })

```

```

with tf.io.TFRecordWriter("my_contacts.tfrecord") as f:
    for _ in range(5):
        f.write(person_example.SerializeToString())

```

Loading and Parsing Examples

```

feature_description = {
    "name": tf.io.FixedLenFeature([], tf.string, default_value=""),
    "id": tf.io.FixedLenFeature([], tf.int64, default_value=0),
    "emails": tf.io.VarLenFeature(tf.string),
}

def parse(serialized_example):
    return tf.io.parse_single_example(serialized_example, feature_description)

dataset = tf.data.TFRecordDataset(["my_contacts.tfrecord"]).map(parse)
for parsed_example in dataset:
    print(parsed_example)

```

```

{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f829147c040>, 'id': <tf.Tensor: shape=(), dtype=
{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f82390756a0>, 'id': <tf.Tensor: shape=(), dtype=
{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f8239068a60>, 'id': <tf.Tensor: shape=(), dtype=
{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f829147b310>, 'id': <tf.Tensor: shape=(), dtype=
{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f829155d850>, 'id': <tf.Tensor: shape=(), dtype=

```

```
tf.sparse.to_dense(parsed_example["emails"], default_value=b"")
```

```
<tf.Tensor: shape=(2,), dtype=string, numpy=array([b'a@b.com', b'c@d.com'], dtype=object)>
```

```
parsed_example["emails"].values
```

```
<tf.Tensor: shape=(2,), dtype=string, numpy=array([b'a@b.com', b'c@d.com'], dtype=object)>
```

```

def parse(serialized_examples):
    return tf.io.parse_example(serialized_examples, feature_description)

dataset = tf.data.TFRecordDataset(["my_contacts.tfrecord"]).batch(2).map(parse)
for parsed_examples in dataset:
    print(parsed_examples) # two examples at a time

```

```

{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f8281729dc0>, 'id': <tf.Tensor: shape=(2,), dtyp
{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f8239068f40>, 'id': <tf.Tensor: shape=(2,), dtyp
{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x7f8239075b50>, 'id': <tf.Tensor: shape=(1,), dtyp

```

```
parsed_examples
```

```

{'emails': <tensorflow.python.framework.sparse_tensor.SparseTensor at 0x7f8239075b50>,
 'id': <tf.Tensor: shape=(1,), dtype=int64, numpy=array([123])>,
 'name': <tf.Tensor: shape=(1,), dtype=string, numpy=array([b'Alice'], dtype=object)>}

```

Extra Material – Storing Images and Tensors in TFRecords

Let's load and display an example image:

```

import matplotlib.pyplot as plt
from sklearn.datasets import load_sample_images

img = load_sample_images()["images"][0]
plt.imshow(img)
plt.axis("off")

```

```
plt.title("Original Image")
plt.show()
```



Now let's create an `Example` protobuf containing the image encoded as JPEG:

```
data = tf.io.encode_jpeg(img)
example_with_image = Example(features=Features(feature={
    "image": Feature(bytes_list=BytesList(value=[data.numpy()]))}))
serialized_example = example_with_image.SerializeToString()
with tf.io.TFRecordWriter("my_image.tfrecord") as f:
    f.write(serialized_example)
```

Finally, let's create a `tf.data` pipeline that will read this TFRecord file, parse each `Example` protobuf (in this case just one), and parse and display the image that the example contains:

```
feature_description = { "image": tf.io.VarLenFeature(tf.string) }

def parse(serialized_example):
    example_with_image = tf.io.parse_single_example(serialized_example,
                                                    feature_description)
    return tf.io.decode_jpeg(example_with_image["image"].values[0])
    # or you can use tf.io.decode_image() instead

dataset = tf.data.TFRecordDataset("my_image.tfrecord").map(parse)
for image in dataset:
    plt.imshow(image)
    plt.axis("off")
    plt.show()
```



Or use `decode_image()` which supports BMP, GIF, JPEG and PNG formats:

Tensors can be serialized and parsed easily using `tf.io.serialize_tensor()` and `tf.io.parse_tensor()`:

```
tensor = tf.constant([[0., 1.], [2., 3.], [4., 5.]])
serialized = tf.io.serialize_tensor(tensor)
serialized
```

[illegible]

```
tf.io.parse_tensor(serialized, out_type=tf.float32)
```

```
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[0., 1.]
```

```
[2., 3.],  
[4., 5.]], dtype=float32)>
```

```
sparse_tensor = parsed_example["emails"]  
serialized_sparse = tf.io.serialize_sparse(sparse_tensor)  
serialized_sparse
```

```
<tf.Tensor: shape=(3,), dtype=string, numpy=  
array([b'\x08\t\x12\x08\x12\x02\x08\x02\x12\x02\x08\x01"\x10\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00',  
      b'\x08\x07\x12\x04\x12\x02\x08\x02"\x10\x07\x07a@b.com@d.com',  
      b'\x08\t\x12\x04\x12\x02\x08\x01"\x08\x02\x00\x00\x00\x00\x00\x00\x00\x00\x00'],  
      dtype=object)>
```

```
BytesList(value=serialized_sparse.numpy())
```

```
value: "\010\t\022\010\022\002\010\002\022\002\010\001\"\020\000\000\000\000\000\000\000\001\000\000\000\000\000\000"  
value: "\010\007\022\004\022\002\010\002\"\020\007\007a@b.com@d.com"  
value: "\010\t\022\004\022\002\010\001\"\010\002\000\000\000\000\000\000\000\000"
```

▼ Handling Lists of Lists Using the `SequenceExample` Protobuf

```
syntax = "proto3";  
  
message FeatureList { repeated Feature feature = 1; };  
message FeatureLists { map<string, FeatureList> feature_list = 1; };  
message SequenceExample {  
    Features context = 1;  
    FeatureLists feature_lists = 2;  
};
```

```
from tensorflow.train import FeatureList, FeatureLists, SequenceExample  
  
context = Features(feature={  
    "author_id": Feature(int64_list=Int64List(value=[123])),  
    "title": Feature(bytes_list=BytesList(value=[b"A", b"desert", b"place", b"."])),  
    "pub_date": Feature(int64_list=Int64List(value=[1623, 12, 25]))  
})  
  
content = [["When", "shall", "we", "three", "meet", "again", "?"],  
           ["In", "thunder", ",", "lightning", ",", "or", "in", "rain", "?"]]  
comments = [["When", "the", "hurlyburly", "'s", "done", "."],  
            ["When", "the", "battle", "'s", "lost", "and", "won", "."]]  
  
def words_to_feature(words):  
    return Feature(bytes_list=BytesList(value=[word.encode("utf-8")  
                                              for word in words]))  
  
content_features = [words_to_feature(sentence) for sentence in content]  
comments_features = [words_to_feature(comment) for comment in comments]  
  
sequence_example = SequenceExample(  
    context=context,  
    feature_lists=FeatureLists(feature_list={  
        "content": FeatureList(feature=content_features),  
        "comments": FeatureList(feature=comments_features)  
    }))
```

```
sequence_example
```

```
context {  
  feature {  
    key: "author_id"  
    value {  
      int64_list {  
        value: 123  
      }  
    }  
  }  
}  
feature {  
  key: "pub_date"  
  value {  
    int64_list {
```

```

        value: 1623
        value: 12
        value: 25
    }
}
}
feature {
  key: "title"
  value {
    bytes_list {
      value: "A"
      value: "desert"
      value: "place"
      value: "."
    }
  }
}
}
feature_lists {
  feature_list {
    key: "comments"
    value {
      feature {
        bytes_list {
          value: "When"
          value: "the"
          value: "hurlyburly"
          value: "\"s"
          value: "done"
          value: "."
        }
      }
      feature {
        bytes_list {
          value: "When"
          value: "the"
          value: "battle"
          value: "\"s"
          value: "lost"
          value: "and"
          value: "won"
          value: "."
        }
      }
    }
  }
}
}

```

```
serialized_sequence_example = sequence_example.SerializeToString()
```

```

context_feature_descriptions = {
  "author_id": tf.io.FixedLenFeature([], tf.int64, default_value=0),
  "title": tf.io.VarLenFeature(tf.string),
  "pub_date": tf.io.FixedLenFeature([3], tf.int64, default_value=[0, 0, 0]),
}
sequence_feature_descriptions = {
  "content": tf.io.VarLenFeature(tf.string),
  "comments": tf.io.VarLenFeature(tf.string),
}

```

```

parsed_context, parsed_feature_lists = tf.io.parse_single_sequence_example(
    serialized_sequence_example, context_feature_descriptions,
    sequence_feature_descriptions)
parsed_content = tf.RaggedTensor.from_sparse(parsed_feature_lists["content"])

```

parsed_context

```
{'title': <tensorflow.python.framework.sparse_tensor.SparseTensor at 0x7f8281d310d0>,
'author_id': <tf.Tensor: shape=(), dtype=int64, numpy=123>,
'pub_date': <tf.Tensor: shape=(3,), dtype=int64, numpy=array([1623, 12, 25])>>}
```

parsed_context["title"].values

```
<tf.Tensor: shape=(4,), dtype=string, numpy=array([b'A', b'desert', b'place', b'.'], dtype=object)>
```

parsed_feature_lists

```
{'comments': <tensorflow.python.framework.sparse_tensor.SparseTensor at 0x7f8281d31be0>,
'content': <tensorflow.python.framework.sparse_tensor.SparseTensor at 0x7f8281d31280>}
```

```
print(tf.RaggedTensor.from_sparse(parsed_feature_lists["content"]))
```

```
<tf.RaggedTensor [[b'When', b'shall', b'we', b'three', b'meet', b'again', b'?'], [b'In', b'thunder', b',', b'lightning', b',', b
```

✓ Keras Preprocessing Layers

✓ The `Normalization` Layer

```
tf.random.set_seed(42) # extra code - ensures reproducibility
norm_layer = tf.keras.layers.Normalization()
model = tf.keras.models.Sequential([
    norm_layer,
    tf.keras.layers.Dense(1)
])
model.compile(loss="mse", optimizer=tf.keras.optimizers.SGD(learning_rate=2e-3))
norm_layer.adapt(X_train) # computes the mean and variance of every feature
model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=5)
```

```
Epoch 1/5
363/363 [=====] - 0s 863us/step - loss: 2.6287 - val_loss: 1.2771
Epoch 2/5
363/363 [=====] - 0s 691us/step - loss: 0.8460 - val_loss: 1.3751
Epoch 3/5
363/363 [=====] - 0s 729us/step - loss: 0.6995 - val_loss: 1.2119
Epoch 4/5
363/363 [=====] - 0s 716us/step - loss: 0.6606 - val_loss: 0.8703
Epoch 5/5
363/363 [=====] - 0s 696us/step - loss: 0.6374 - val_loss: 0.6106
<keras.callbacks.History at 0x7f8241cba1f0>
```

```
norm_layer = tf.keras.layers.Normalization()
norm_layer.adapt(X_train)
X_train_scaled = norm_layer(X_train)
X_valid_scaled = norm_layer(X_valid)
```

```
tf.random.set_seed(42) # extra code - ensures reproducibility
model = tf.keras.models.Sequential([tf.keras.layers.Dense(1)])
model.compile(loss="mse", optimizer=tf.keras.optimizers.SGD(learning_rate=2e-3))
model.fit(X_train_scaled, y_train, epochs=5,
          validation_data=(X_valid_scaled, y_valid))
```

```
Epoch 1/5
363/363 [=====] - 0s 806us/step - loss: 2.6287 - val_loss: 1.2771
Epoch 2/5
363/363 [=====] - 0s 642us/step - loss: 0.8460 - val_loss: 1.3751
Epoch 3/5
363/363 [=====] - 0s 647us/step - loss: 0.6995 - val_loss: 1.2119
Epoch 4/5
363/363 [=====] - 0s 669us/step - loss: 0.6606 - val_loss: 0.8703
Epoch 5/5
363/363 [=====] - 0s 651us/step - loss: 0.6374 - val_loss: 0.6106
<keras.callbacks.History at 0x7f8272695400>
```

```
final_model = tf.keras.Sequential([norm_layer, model])
X_new = X_test[:3] # pretend we have a few new instances (unscaled)
y_pred = final_model(X_new) # preprocesses the data and makes predictions
```

```
y_pred
```

```
<tf.Tensor: shape=(3, 1), dtype=float32, numpy=
array([[1.0205517],
       [1.5699625],
       [2.460654 ]], dtype=float32)>
```

```
# extra code - creates a dataset to demo applying the norm_layer using map()
dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(5)
```

```
dataset = dataset.map(lambda X, y: (norm_layer(X), y))
```

```
list(dataset.take(1)) # extra code - shows the first batch
```

```
[<tf.Tensor: shape=(5, 8), dtype=float32, numpy=
array([[ -0.1939791 , -1.0778134 , -0.9433871 ,  0.0148516 ,  0.02073434,
        -0.572917  ,  0.92925584, -1.4221287 ],
       [ 0.7519827 , -1.8688954 ,  0.40547717, -0.23327832,  1.8614666 ,
         0.20516507, -0.9165531 ,  1.0966995 ],
       [-0.41469136,  0.02970134,  0.8180875 ,  1.0567819 , -0.08786613,
        -0.29983336,  1.3087229 , -1.6970023 ],
       [ 1.7188951 , -1.315138 ,  0.32664284, -0.21955258, -0.337921 ,
        -0.11146677, -0.9821399 ,  0.9417729 ],
       [-0.96207225, -1.2360299 , -0.05625898, -0.03124549,  1.709061 ,
        -0.30257043, -0.8041173 ,  1.3265921 ]], dtype=float32)>,
 <tf.Tensor: shape=(5, 1), dtype=float64, numpy=
array([[1.442],
       [1.687],
       [1.621],
       [2.621],
       [0.956]])>>]
```

```
class MyNormalization(tf.keras.layers.Layer):
    def adapt(self, X):
        self.mean_ = np.mean(X, axis=0, keepdims=True)
        self.std_ = np.std(X, axis=0, keepdims=True)

    def call(self, inputs):
        eps = tf.keras.backend.epsilon() # a small smoothing term
        return (inputs - self.mean_) / (self.std_ + eps)
```

```
my_norm_layer = MyNormalization()
my_norm_layer.adapt(X_train)
X_train_scaled = my_norm_layer(X_train)
```

✓ The Discretization Layer

```
age = tf.constant([[10.], [93.], [57.], [18.], [37.], [5.]])
discretize_layer = tf.keras.layers.Discretization(bin_boundaries=[18., 50.])
age_categories = discretize_layer(age)
age_categories
```

```
<tf.Tensor: shape=(6, 1), dtype=int64, numpy=
array([[0],
       [2],
       [2],
       [1],
       [1],
       [0]])>
```

```
discretize_layer = tf.keras.layers.Discretization(num_bins=3)
discretize_layer.adapt(age)
age_categories = discretize_layer(age)
age_categories
```

```
<tf.Tensor: shape=(6, 1), dtype=int64, numpy=
array([[1],
       [2],
       [2],
       [1],
       [2],
       [0]])>
```

✓ The CategoryEncoding Layer

```
onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3)
onehot_layer(age_categories)
```

```
<tf.Tensor: shape=(6, 3), dtype=float32, numpy=
array([[0., 1., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.]], dtype=float32)>
```

```
two_age_categories = np.array([[1, 0], [2, 2], [2, 0]])
onehot_layer(two_age_categories)
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[1., 1., 0.],
       [0., 0., 1.],
       [1., 0., 1.]], dtype=float32)>
```

```
onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3, output_mode="count")
onehot_layer(two_age_categories)
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[1., 1., 0.],
       [0., 0., 2.],
       [1., 0., 1.]], dtype=float32)>
```

```
onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3 + 3)
onehot_layer(two_age_categories + [0, 3]) # adds 3 to the second feature
```

```
<tf.Tensor: shape=(3, 6), dtype=float32, numpy=
array([[0., 1., 0., 1., 0., 0.],
       [0., 0., 1., 0., 0., 1.],
       [0., 0., 1., 1., 0., 0.]], dtype=float32)>
```

```
# extra code - shows another way to one-hot encode each feature separately
onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3,
                                                output_mode="one_hot")
tf.keras.layers.concatenate([onehot_layer(cat)
                             for cat in tf.transpose(two_age_categories)])
```

```
<tf.Tensor: shape=(3, 6), dtype=float32, numpy=
array([[0., 1., 0., 1., 0., 0.],
       [0., 0., 1., 0., 0., 1.],
       [0., 0., 1., 1., 0., 0.]], dtype=float32)>
```

```
# extra code - shows another way to do this, using tf.one_hot() and Flatten
tf.keras.layers.Flatten()(tf.one_hot(two_age_categories, depth=3))
```

```
<tf.Tensor: shape=(3, 6), dtype=float32, numpy=
array([[0., 1., 0., 1., 0., 0.],
       [0., 0., 1., 0., 0., 1.],
       [0., 0., 1., 1., 0., 0.]], dtype=float32)>
```

▼ The `StringLookup` Layer

```
cities = ["Auckland", "Paris", "Paris", "San Francisco"]
str_lookup_layer = tf.keras.layers.StringLookup()
str_lookup_layer.adapt(cities)
str_lookup_layer([["Paris"], ["Auckland"], ["Auckland"], ["Montreal"]])
```

```
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=
array([[1],
       [3],
       [3],
       [0]])>
```

```
str_lookup_layer = tf.keras.layers.StringLookup(num_oov_indices=5)
str_lookup_layer.adapt(cities)
str_lookup_layer([["Paris"], ["Auckland"], ["Foo"], ["Bar"], ["Baz"]])
```

```
<tf.Tensor: shape=(5, 1), dtype=int64, numpy=
array([[5],
       [7],
       [4],
       [3],
       [4]])>
```

```
str_lookup_layer = tf.keras.layers.StringLookup(output_mode="one_hot")
str_lookup_layer.adapt(cities)
str_lookup_layer([["Paris"], ["Auckland"], ["Auckland"], ["Montreal"]])
```

```
WARNING:tensorflow:5 out of the last 367 calls to <function PreprocessingLayer.make_adapt_function.<locals>.adapt_step at 0x7f82
<tf.Tensor: shape=(4, 4), dtype=float32, numpy=
array([[0., 1., 0., 0.],
       [0., 0., 0., 1.],
```



```
[0., 0., 0., 1.],
[1., 0., 0., 0.]], dtype=float32)>
```

```
# extra code - an example using the IntegerLookup layer
ids = [123, 456, 789]
int_lookup_layer = tf.keras.layers.IntegerLookup()
int_lookup_layer.adapt(ids)
int_lookup_layer([[123], [456], [123], [111]])
```

```
WARNING:tensorflow:6 out of the last 368 calls to <function PreprocessingLayer.make_adapt_function.<locals>.adapt_step at 0x7f82
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=
array([[3],
       [2],
       [3],
       [0]])>
```

✓ The Hashing Layer

```
hashing_layer = tf.keras.layers.Hashing(num_bins=10)
hashing_layer([["Paris"], ["Tokyo"], ["Auckland"], ["Montreal"]])
```

```
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=
array([[0],
       [1],
       [9],
       [1]])>
```

✓ Encoding Categorical Features Using Embeddings

```
tf.random.set_seed(42)
embedding_layer = tf.keras.layers.Embedding(input_dim=5, output_dim=2)
embedding_layer(np.array([2, 4, 2]))
```

```
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[ -0.04663396,  0.01846724],
       [ -0.02736737, -0.02768031],
       [ -0.04663396,  0.01846724]], dtype=float32)>
```

Warning: there's a bug in Keras 2.8.0 ([issue #16101](#)) which prevents using a `StringLookup` layer as the first layer of a `Sequential` model. Luckily, there's a simple workaround: just add an `InputLayer` as the first layer.

```
tf.random.set_seed(42)
ocean_prox = ["<1H OCEAN", "INLAND", "NEAR OCEAN", "NEAR BAY", "ISLAND"]
str_lookup_layer = tf.keras.layers.StringLookup()
str_lookup_layer.adapt(ocean_prox)
lookup_and_embed = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=[], dtype=tf.string), # WORKAROUND
    str_lookup_layer,
    tf.keras.layers.Embedding(input_dim=str_lookup_layer.vocabulary_size(),
                              output_dim=2)
])
lookup_and_embed(np.array(["<1H OCEAN", "ISLAND", "<1H OCEAN"]))
```

```
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[ -0.01896119,  0.02223358],
       [  0.02401174,  0.03724445],
       [ -0.01896119,  0.02223358]], dtype=float32)>
```

```
# extra code - set seeds and generates fake random data
# (feel free to load the real dataset if you prefer)
tf.random.set_seed(42)
np.random.seed(42)
X_train_num = np.random.rand(10_000, 8)
X_train_cat = np.random.choice(ocean_prox, size=10_000).astype(object)
y_train = np.random.rand(10_000, 1)
X_valid_num = np.random.rand(2_000, 8)
X_valid_cat = np.random.choice(ocean_prox, size=2_000).astype(object)
y_valid = np.random.rand(2_000, 1)

num_input = tf.keras.layers.Input(shape=[8], name="num")
```

```
cat_input = tf.keras.layers.Input(shape=[], dtype=tf.string, name="cat")
cat_embeddings = lookup_and_embed(cat_input)
encoded_inputs = tf.keras.layers.concatenate([num_input, cat_embeddings])
outputs = tf.keras.layers.Dense(1)(encoded_inputs)
model = tf.keras.models.Model(inputs=[num_input, cat_input], outputs=[outputs])
model.compile(loss="mse", optimizer="sgd")
history = model.fit((X_train_num, X_train_cat), y_train, epochs=5,
                    validation_data=(X_valid_num, X_valid_cat), y_valid))
```

```
Epoch 1/5
313/313 [=====] - 0s 903us/step - loss: 0.1491 - val_loss: 0.1188
Epoch 2/5
313/313 [=====] - 0s 723us/step - loss: 0.1069 - val_loss: 0.0967
Epoch 3/5
313/313 [=====] - 0s 667us/step - loss: 0.0924 - val_loss: 0.0886
Epoch 4/5
313/313 [=====] - 0s 677us/step - loss: 0.0870 - val_loss: 0.0856
Epoch 5/5
313/313 [=====] - 0s 671us/step - loss: 0.0849 - val_loss: 0.0843
```

```
# extra code - shows that the model can also be trained using a tf.data.Dataset
train_set = tf.data.Dataset.from_tensor_slices(
    ((X_train_num, X_train_cat), y_train)).batch(32)
valid_set = tf.data.Dataset.from_tensor_slices(
    ((X_valid_num, X_valid_cat), y_valid)).batch(32)
history = model.fit(train_set, epochs=5,
                    validation_data=valid_set)
```

```
Epoch 1/5
313/313 [=====] - 1s 1ms/step - loss: 0.0839 - val_loss: 0.0838
Epoch 2/5
313/313 [=====] - 0s 1ms/step - loss: 0.0835 - val_loss: 0.0835
Epoch 3/5
313/313 [=====] - 0s 1ms/step - loss: 0.0832 - val_loss: 0.0833
Epoch 4/5
313/313 [=====] - 0s 1ms/step - loss: 0.0831 - val_loss: 0.0832
Epoch 5/5
313/313 [=====] - 0s 1ms/step - loss: 0.0830 - val_loss: 0.0831
```

✧ Text Preprocessing

```
train_data = ["To be", "!(to be)", "That's the question", "Be, be, be."]
text_vec_layer = tf.keras.layers.TextVectorization()
text_vec_layer.adapt(train_data)
text_vec_layer(["Be good!", "Question: be or be?"])
```

```
<tf.Tensor: shape=(2, 4), dtype=int64, numpy=
array([[2, 1, 0, 0],
       [6, 2, 1, 2]])>
```

```
text_vec_layer = tf.keras.layers.TextVectorization(ragged=True)
text_vec_layer.adapt(train_data)
text_vec_layer(["Be good!", "Question: be or be?"])
```

```
<tf.RaggedTensor [[2, 1], [6, 2, 1, 2]]>
```

```
text_vec_layer = tf.keras.layers.TextVectorization(output_mode="tf_idf")
text_vec_layer.adapt(train_data)
text_vec_layer(["Be good!", "Question: be or be?"])
```

```
<tf.Tensor: shape=(2, 6), dtype=float32, numpy=
array([[0.96725637, 0.6931472, 0.          , 0.          ,
        0.          , 0.          ],
       [0.96725637, 1.3862944, 0.          , 0.          ,
        1.0986123, 0.          ]], dtype=float32)>
```

```
2 * np.log(1 + 4 / (1 + 3))
```

```
1.3862943611198906
```

```
1 * np.log(1 + 4 / (1 + 1))
```

```
1.0986122886681098
```

Using Pretrained Language Model Components

```
import tensorflow_hub as hub
```

```
hub_layer = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim50/2")  
sentence_embeddings = hub_layer(tf.constant(["To be", "Not to be"]))  
sentence_embeddings.numpy().round(2)
```

```
array([[ -0.25,  0.28,  0.01,  0.1 ,  0.14,  0.16,  0.25,  0.02,  0.07,  
         0.13, -0.19,  0.06, -0.04, -0.07,  0. , -0.08, -0.14, -0.16,  
         0.02, -0.24,  0.16, -0.16, -0.03,  0.03, -0.14,  0.03, -0.09,  
        -0.04, -0.14, -0.19,  0.07,  0.15,  0.18, -0.23, -0.07, -0.08,  
         0.01, -0.01,  0.09,  0.14, -0.03,  0.03,  0.08,  0.1 , -0.01,  
        -0.03, -0.07, -0.1 ,  0.05,  0.31],  
       [-0.2 ,  0.2 , -0.08,  0.02,  0.19,  0.05,  0.22, -0.09,  0.02,  
         0.19, -0.02, -0.14, -0.2 , -0.04,  0.01, -0.07, -0.22, -0.1 ,  
         0.16, -0.44,  0.31, -0.1 ,  0.23,  0.15, -0.05,  0.15, -0.13,  
        -0.04, -0.08, -0.16, -0.1 ,  0.13,  0.13, -0.18, -0.04,  0.03,  
        -0.1 , -0.07,  0.07,  0.03, -0.08,  0.02,  0.05,  0.07, -0.14,  
        -0.1 , -0.18, -0.13, -0.04,  0.15]], dtype=float32)
```

Image Preprocessing Layers

```
from sklearn.datasets import load_sample_images
```

```
images = load_sample_images()["images"]  
crop_image_layer = tf.keras.layers.CenterCrop(height=100, width=100)  
cropped_images = crop_image_layer(images)
```

```
plt.imshow(images[0])  
plt.axis("off")  
plt.show()
```



```
plt.imshow(cropped_images[0] / 255)  
plt.axis("off")  
plt.show()
```



TensorFlow Datasets

```
import tensorflow_datasets as tfds
```

```
datasets = tfds.load(name="mnist")  
mnist_train, mnist_test = datasets["train"], datasets["test"]
```

```
for batch in mnist_train.shuffle(10_000, seed=42).batch(32).prefetch(1):
    images = batch["image"]
    labels = batch["label"]
    # [...] do something with the images and labels
```

```
mnist_train = mnist_train.shuffle(10_000, seed=42).batch(32)
mnist_train = mnist_train.map(lambda items: (items["image"], items["label"]))
mnist_train = mnist_train.prefetch(1)
```

```
train_set, valid_set, test_set = tfds.load(
    name="mnist",
    split=["train[:90%]", "train[90%:]", "test"],
    as_supervised=True
)
train_set = train_set.shuffle(10_000, seed=42).batch(32).prefetch(1)
valid_set = valid_set.batch(32).cache()
test_set = test_set.batch(32).cache()
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=5)
test_loss, test_accuracy = model.evaluate(test_set)
```

```
Epoch 1/5
1688/1688 [=====] - 2s 1ms/step - loss: 9.6765 - accuracy: 0.8348 - val_loss: 5.8894 - val_accuracy: 0.
Epoch 2/5
1688/1688 [=====] - 1s 796us/step - loss: 5.6335 - accuracy: 0.8785 - val_loss: 5.1325 - val_accuracy:
Epoch 3/5
1688/1688 [=====] - 1s 793us/step - loss: 5.0494 - accuracy: 0.8832 - val_loss: 5.3470 - val_accuracy:
Epoch 4/5
1688/1688 [=====] - 1s 767us/step - loss: 4.8245 - accuracy: 0.8867 - val_loss: 5.2491 - val_accuracy:
Epoch 5/5
1688/1688 [=====] - 1s 765us/step - loss: 4.6808 - accuracy: 0.8871 - val_loss: 5.1136 - val_accuracy:
313/313 [=====] - 0s 769us/step - loss: 4.6993 - accuracy: 0.8975
```

✓ Exercises

1. to 8.

1. Ingesting a large dataset and preprocessing it efficiently can be a complex engineering challenge. The Data API makes it fairly simple. It offers many features, including loading data from various sources (such as text or binary files), reading data in parallel from multiple sources, transforming it, interleaving the records, shuffling the data, batching it, and prefetching it.
2. Splitting a large dataset into multiple files makes it possible to shuffle it at a coarse level before shuffling it at a finer level using a shuffling buffer. It also makes it possible to handle huge datasets that do not fit on a single machine. It's also simpler to manipulate thousands of small files rather than one huge file; for example, it's easier to split the data into multiple subsets. Lastly, if the data is split across multiple files spread across multiple servers, it is possible to download several files from different servers simultaneously, which improves the bandwidth usage.
3. You can use TensorBoard to visualize profiling data: if the GPU is not fully utilized then your input pipeline is likely to be the bottleneck. You can fix it by making sure it reads and preprocesses the data in multiple threads in parallel, and ensuring it prefetches a few batches. If this is insufficient to get your GPU to 100% usage during training, make sure your preprocessing code is optimized. You can also try saving the dataset into multiple TFRecord files, and if necessary perform some of the preprocessing ahead of time so that it does not need to be done on the fly during training (TF Transform can help with this). If necessary, use a machine with more CPU and RAM, and ensure that the GPU bandwidth is large enough.
4. A TFRecord file is composed of a sequence of arbitrary binary records: you can store absolutely any binary data you want in each record. However, in practice most TFRecord files contain sequences of serialized protocol buffers. This makes it possible to benefit from the advantages of protocol buffers, such as the fact that they can be read easily across multiple platforms and languages and their definition can be updated later in a backward-compatible way.
5. The `Example` protobuf format has the advantage that TensorFlow provides some operations to parse it (the `tf.io.parse_*example()` functions) without you having to define your own format. It is sufficiently flexible to represent instances in most datasets. However, if it does not cover your use case, you can define your own protocol buffer, compile it using `protoc`

(setting the `--descriptor_set_out` and `--include_imports` arguments to export the protobuf descriptor), and use the `tf.io.decode_proto()` function to parse the serialized protobufs (see the "Custom protobuf" section of the notebook for an example). It's more complicated, and it requires deploying the descriptor along with the model, but it can be done.

6. When using TFRecords, you will generally want to activate compression if the TFRecord files will need to be downloaded by the training script, as compression will make files smaller and thus reduce download time. But if the files are located on the same machine as the training script, it's usually preferable to leave compression off, to avoid wasting CPU for decompression.
7. Let's look at the pros and cons of each preprocessing option:
 - If you preprocess the data when creating the data files, the training script will run faster, since it will not have to perform preprocessing on the fly. In some cases, the preprocessed data will also be much smaller than the original data, so you can save some space and speed up downloads. It may also be helpful to materialize the preprocessed data, for example to inspect it or archive it. However, this approach has a few cons. First, it's not easy to experiment with various preprocessing logics if you need to generate a preprocessed dataset for each variant. Second, if you want to perform data augmentation, you have to materialize many variants of your dataset, which will use a large amount of disk space and take a lot of time to generate. Lastly, the trained model will expect preprocessed data, so you will have to add preprocessing code in your application before it calls the model. There's a risk of code duplication and preprocessing mismatch in this case.
 - If the data is preprocessed with the `tf.data` pipeline, it's much easier to tweak the preprocessing logic and apply data augmentation. Also, `tf.data` makes it easy to build highly efficient preprocessing pipelines (e.g., with multithreading and prefetching). However, preprocessing the data this way will slow down training. Moreover, each training instance will be preprocessed once per epoch rather than just once if the data was preprocessed when creating the data files. Well, unless the dataset fits in RAM and you can cache it using the dataset's `cache()` method. Lastly, the trained model will still expect preprocessed data. But if you use preprocessing layers in your `tf.data` pipeline to handle the preprocessing step, then you can just reuse these layers in your final model (adding them after training), to avoid code duplication and preprocessing mismatch.
 - If you add preprocessing layers to your model, you will only have to write the preprocessing code once for both training and inference. If your model needs to be deployed to many different platforms, you will not need to write the preprocessing code multiple times. Plus, you will not run the risk of using the wrong preprocessing logic for your model, since it will be part of the model. On the downside, preprocessing the data on the fly during training will slow things down, and each instance will be preprocessed once per epoch.

8. Let's look at how to encode categorical text features and text:

- To encode a categorical feature that has a natural order, such as a movie rating (e.g., "bad," "average," "good"), the simplest option is to use ordinal encoding: sort the categories in their natural order and map each category to its rank (e.g., "bad" maps to 0, "average" maps to 1, and "good" maps to 2). However, most categorical features don't have such a natural order. For example, there's no natural order for professions or countries. In this case, you can use one-hot encoding, or embeddings if there are many categories. With Keras, the `StringLookup` layer can be used for ordinal encoding (using the default `output_mode="int"`), or one-hot encoding (using `output_mode="one_hot"`). It can also perform multi-hot encoding (using `output_mode="multi_hot"`) if you want to encode multiple categorical text features together, assuming they share the same categories and it doesn't matter which feature contributed which category. For trainable embeddings, you must first use the `StringLookup` layer to produce an ordinal encoding, then use the `Embedding` layer.
- For text, the `TextVectorization` layer is easy to use and it can work well for simple tasks, or you can use TF Text for more advanced features. However, you'll often want to use pretrained language models, which you can obtain using tools like TF Hub or Hugging Face's Transformers library. These last two options are discussed in Chapter 16.

9.

a.

Exercise: Load the Fashion MNIST dataset (introduced in Chapter 10); split it into a training set, a validation set, and a test set; shuffle the training set; and save each dataset to multiple TFRecord files. Each record should be a serialized `Example` protobuf with two features: the serialized image (use `tf.io.serialize_tensor()` to serialize each image), and the label. Note: for large images, you could use `tf.io.encode_jpeg()` instead. This would save a lot of space, but it would lose a bit of image quality.

```
(X_train_full, y_train_full), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
```

```
tf.random.set_seed(42)
train_set = tf.data.Dataset.from_tensor_slices((X_train, y_train))
train_set = train_set.shuffle(len(X_train), seed=42)
```



```

        "image": tf.io.FixedLenFeature([], tf.string, default_value=""),
        "label": tf.io.FixedLenFeature([], tf.int64, default_value=-1)
    }
    example = tf.io.parse_single_example(tfreCORD, feature_descriptions)
    image = tf.io.parse_tensor(example["image"], out_type=tf.uint8)
    #image = tf.io.decode_jpeg(example["image"])
    image = tf.reshape(image, shape=[28, 28])
    return image, example["label"]

def mnist_dataset(filepaths, n_read_threads=5, shuffle_buffer_size=None,
                  n_parse_threads=5, batch_size=32, cache=True):
    dataset = tf.data.TFRecordDataset(filepaths,
                                     num_parallel_reads=n_read_threads)

    if cache:
        dataset = dataset.cache()
    if shuffle_buffer_size:
        dataset = dataset.shuffle(shuffle_buffer_size)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
    dataset = dataset.batch(batch_size)
    return dataset.prefetch(1)

```

```

train_set = mnist_dataset(train_filepaths, shuffle_buffer_size=60000)
valid_set = mnist_dataset(valid_filepaths)
test_set = mnist_dataset(test_filepaths)

```

```

for X, y in train_set.take(1):
    for i in range(5):
        plt.subplot(1, 5, i + 1)
        plt.imshow(X[i].numpy(), cmap="binary")
        plt.axis("off")
        plt.title(str(y[i].numpy()))

```



```

tf.random.set_seed(42)

standardization = tf.keras.layers.Normalization(input_shape=[28, 28])

sample_image_batches = train_set.take(100).map(lambda image, label: image)
sample_images = np.concatenate(list(sample_image_batches.as_numpy_iterator()),
                               axis=0).astype(np.float32)
standardization.adapt(sample_images)

model = tf.keras.Sequential([
    standardization,
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="nadam", metrics=["accuracy"])

```

```

from datetime import datetime

logs = Path() / "my_logs" / "run_" / datetime.now().strftime("%Y%m%d_%H%M%S")

tensorboard_cb = tf.keras.callbacks.TensorBoard(
    log_dir=logs, histogram_freq=1, profile_batch=10)

model.fit(train_set, epochs=5, validation_data=valid_set,
          callbacks=[tensorboard_cb])

```

```

2022-02-20 15:30:49.689831: I tensorflow/core/profiler/lib/profiler_session.cc:110] Profiler session initializing.
2022-02-20 15:30:49.689858: I tensorflow/core/profiler/lib/profiler_session.cc:125] Profiler session started.
2022-02-20 15:30:49.691427: I tensorflow/core/profiler/lib/profiler_session.cc:143] Profiler session tear down.
Epoch 1/5
59/Unknown - 1s 3ms/step - loss: 0.9230 - accuracy: 0.68172022-02-20 15:30:50.428921: I tensorflow/core/profiler/lib/profil
2022-02-20 15:30:50.428945: I tensorflow/core/profiler/lib/profiler_session.cc:125] Profiler session started.
2022-02-20 15:30:50.433359: I tensorflow/core/profiler/lib/profiler_session.cc:67] Profiler session collecting data.
2022-02-20 15:30:50.446608: I tensorflow/core/profiler/lib/profiler_session.cc:143] Profiler session tear down.
2022-02-20 15:30:50.461272: I tensorflow/core/profiler/rpc/client/save_profile.cc:136] Creating directory: my_logs/run_/20220220

```

```

2022-02-20 15:30:50.465450: I tensorflow/core/profiler/rpc/client/save_profile.cc:142] Dumped gzipped tool data for trace.json.g
2022-02-20 15:30:50.480245: I tensorflow/core/profiler/rpc/client/save_profile.cc:136] Creating directory: my_logs/run_/20220220

2022-02-20 15:30:50.480582: I tensorflow/core/profiler/rpc/client/save_profile.cc:142] Dumped gzipped tool data for memory_profi
2022-02-20 15:30:50.482034: I tensorflow/core/profiler/rpc/client/capture_profile.cc:251] Creating directory: my_logs/run_/20220
Dumped tool data for xplane.pb to my_logs/run_/20220220_153049/plugins/profile/2022_02_20_15_30_50/kiwimac.xplane.pb
Dumped tool data for overview_page.pb to my_logs/run_/20220220_153049/plugins/profile/2022_02_20_15_30_50/kiwimac.overview_page.
Dumped tool data for input_pipeline.pb to my_logs/run_/20220220_153049/plugins/profile/2022_02_20_15_30_50/kiwimac.input_pipel
Dumped tool data for tensorflow_stats.pb to my_logs/run_/20220220_153049/plugins/profile/2022_02_20_15_30_50/kiwimac.tensorflow_
Dumped tool data for kernel_stats.pb to my_logs/run_/20220220_153049/plugins/profile/2022_02_20_15_30_50/kiwimac.kernel_stats.pb

1719/1719 [=====] - 5s 2ms/step - loss: 0.4437 - accuracy: 0.8402 - val_loss: 0.3649 - val_accuracy: 0.
Epoch 2/5
1719/1719 [=====] - 4s 2ms/step - loss: 0.3333 - accuracy: 0.8775 - val_loss: 0.3346 - val_accuracy: 0.
Epoch 3/5
1719/1719 [=====] - 4s 2ms/step - loss: 0.2970 - accuracy: 0.8905 - val_loss: 0.3235 - val_accuracy: 0.
Epoch 4/5
1719/1719 [=====] - 4s 2ms/step - loss: 0.2723 - accuracy: 0.8995 - val_loss: 0.3308 - val_accuracy: 0.
Epoch 5/5
1719/1719 [=====] - 4s 2ms/step - loss: 0.2534 - accuracy: 0.9047 - val_loss: 0.3174 - val_accuracy: 0.
<keras.callbacks.History at 0x7fa3e08af370>

```

```

%load_ext tensorboard
%tensorboard --logdir=./my_logs

```

```

The tensorboard extension is already loaded. To reload it, use:
%reload_ext tensorboard

```

10.

Exercise: In this exercise you will download a dataset, split it, create a `tf.data.Dataset` to load it and preprocess it efficiently, then build and train a binary classification model containing an `Embedding` layer.

a.

Exercise: Download the [Large Movie Review Dataset](#), which contains 50,000 movies reviews from the [Internet Movie Database](#). The data is organized in two directories, `train` and `test`, each containing a `pos` subdirectory with 12,500 positive reviews and a `neg` subdirectory with 12,500 negative reviews. Each review is stored in a separate text file. There are other files and folders (including preprocessed bag-of-words), but we will ignore them in this exercise.

Warning: in recent TF versions, the `get_file()` function wraps the extracted directory in a another directory with `"_extracted"` in its name:

```
from pathlib import Path

root = "https://ai.stanford.edu/~amaas/data/sentiment/"
filename = "aclImdb_v1.tar.gz"
filepath = tf.keras.utils.get_file(filename, root + filename, extract=True,
                                   cache_dir=".")

if "_extracted" in filepath:
    path = Path(filepath) / "aclImdb"
else:
    path = Path(filepath).with_name("aclImdb")
```

Let's define a `tree()` function to view the structure of the `aclImdb` directory:

```
def tree(path, level=0, indent=4, max_files=3):
    if level == 0:
        print(f"{path}/")
        level += 1
    sub_paths = sorted(path.iterdir())
    sub_dirs = [sub_path for sub_path in sub_paths if sub_path.is_dir()]
    filepaths = [sub_path for sub_path in sub_paths if not sub_path.is_dir()]
    indent_str = " " * indent * level
    for sub_dir in sub_dirs:
        print(f"{indent_str}{sub_dir.name}/")
        tree(sub_dir, level + 1, indent)
    for filepath in filepaths[:max_files]:
        print(f"{indent_str}{filepath.name}")
    if len(filepaths) > max_files:
        print(f"{indent_str}...")
```

`tree(path)`

```
datasets/aclImdb/
  test/
    neg/
      0_2.txt
      10000_4.txt
      10001_1.txt
      ...
    pos/
      0_10.txt
      10000_7.txt
      10001_9.txt
      ...
    labeledBow.feats
    urls_neg.txt
    urls_pos.txt
  train/
    neg/
      0_3.txt
      10000_4.txt
      10001_4.txt
      ...
    pos/
      0_9.txt
      10000_8.txt
      10001_10.txt
      ...
    unsup/
      0_0.txt
      10000_0.txt
      10001_0.txt
      ...
    labeledBow.feats
    unsupBow.feats
```

```
urls_neg.txt
...
README
imdb.vocab
imdbEr.txt
```

```
def review_paths(dirpath):
    return [str(path) for path in dirpath.glob("*.txt")]

train_pos = review_paths(path / "train" / "pos")
train_neg = review_paths(path / "train" / "neg")
test_valid_pos = review_paths(path / "test" / "pos")
test_valid_neg = review_paths(path / "test" / "neg")

len(train_pos), len(train_neg), len(test_valid_pos), len(test_valid_neg)

(12500, 12500, 12500, 12500)
```

✓ b.

Exercise: Split the test set into a validation set (15,000) and a test set (10,000).

```
np.random.shuffle(test_valid_pos)

test_pos = test_valid_pos[:5000]
test_neg = test_valid_neg[:5000]
valid_pos = test_valid_pos[5000:]
valid_neg = test_valid_neg[5000:]
```

✓ c.

Exercise: Use tf.data to create an efficient dataset for each set.

Since the dataset fits in memory, we can just load all the data using pure Python code and use

`tf.data.Dataset.from_tensor_slices()`:

```
def imdb_dataset(filepaths_positive, filepaths_negative):
    reviews = []
    labels = []
    for filepath, label in ((filepaths_negative, 0), (filepaths_positive, 1)):
        for filepath in filepaths:
            with open(filepath) as review_file:
                reviews.append(review_file.read())
                labels.append(label)
    return tf.data.Dataset.from_tensor_slices(
        (tf.constant(reviews), tf.constant(labels)))
```

```
for X, y in imdb_dataset(train_pos, train_neg).take(3):
    print(X)
    print(y)
    print()
```

```
tf.Tensor(b"Working with one of the best Shakespeare sources, this film manages to be creditable to it's source, whilst still ap
tf.Tensor(0, shape=(), dtype=int32)
```

```
tf.Tensor(b'Well...tremors I, the original started off in 1990 and i found the movie quite enjoyable to watch. however, they pro
tf.Tensor(0, shape=(), dtype=int32)
```

```
tf.Tensor(b"Ouch! This one was a bit painful to sit through. It has a cute and amusing premise, but it all goes to hell from the
tf.Tensor(0, shape=(), dtype=int32)
```

```
%timeit -r1 for X, y in imdb_dataset(train_pos, train_neg).repeat(10): pass
```

```
29.7 s ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

It takes about 17 seconds to load the dataset and go through it 10 times.

But let's pretend the dataset does not fit in memory, just to make things more interesting. Luckily, each review fits on just one line (they use `<br />` to indicate line breaks), so we can read the reviews using a `TextLineDataset`. If they didn't we would have to preprocess the input files (e.g., converting them to TFRecords). For very large datasets, it would make sense to use a tool like Apache Beam for that.

```
def imdb_dataset(filepaths_positive, filepaths_negative, n_read_threads=5):
    dataset_neg = tf.data.TextLineDataset(filepaths_negative,
                                          num_parallel_reads=n_read_threads)
    dataset_neg = dataset_neg.map(lambda review: (review, 0))
    dataset_pos = tf.data.TextLineDataset(filepaths_positive,
                                          num_parallel_reads=n_read_threads)
    dataset_pos = dataset_pos.map(lambda review: (review, 1))
    return tf.data.Dataset.concatenate(dataset_pos, dataset_neg)
```

```
%timeit -r1 for X, y in imdb_dataset(train_pos, train_neg).repeat(10): pass
```

```
27.7 s ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

Now it takes about 33 seconds to go through the dataset 10 times. That's much slower, essentially because the dataset is not cached in RAM, so it must be reloaded at each epoch. If you add `.cache()` just before `.repeat(10)`, you will see that this implementation will be about as fast as the previous one.

```
%timeit -r1 for X, y in imdb_dataset(train_pos, train_neg).cache().repeat(10): pass
```

```
20.6 s ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

```
batch_size = 32
```

```
train_set = imdb_dataset(train_pos, train_neg).shuffle(25000, seed=42)
train_set = train_set.batch(batch_size).prefetch(1)
valid_set = imdb_dataset(valid_pos, valid_neg).batch(batch_size).prefetch(1)
test_set = imdb_dataset(test_pos, test_neg).batch(batch_size).prefetch(1)
```

▼ d.

Exercise: Create a binary classification model, using a `TextVectorization` layer to preprocess each review.

Let's create a `TextVectorization` layer and adapt it to the full IMDB training set (if the training set did not fit in RAM, we could just use a smaller sample of the training set by calling `train_set.take(500)`). Let's use TF-IDF for now.

```
max_tokens = 1000
sample_reviews = train_set.map(lambda review, label: review)
text_vectorization = tf.keras.layers.TextVectorization(
    max_tokens=max_tokens, output_mode="tf-idf")
text_vectorization.adapt(sample_reviews)
```

Good! Now let's take a look at the first 10 words in the vocabulary:

```
text_vectorization.get_vocabulary()[:10]
```

```
['[UNK]', 'the', 'and', 'a', 'of', 'to', 'is', 'in', 'it', 'i']
```

These are the most common words in the reviews.

We're ready to train the model!

```
tf.random.set_seed(42)
model = tf.keras.Sequential([
    text_vectorization,
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(loss="binary_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
model.fit(train_set, epochs=5, validation_data=valid_set)
```

```
Epoch 1/5
782/782 [=====] - 4s 4ms/step - loss: 0.4521 - accuracy: 0.8189 - val_loss: 0.3894 - val_accuracy: 0.84
Epoch 2/5
782/782 [=====] - 4s 4ms/step - loss: 0.3608 - accuracy: 0.8537 - val_loss: 0.7081 - val_accuracy: 0.76
Epoch 3/5
782/782 [=====] - 4s 4ms/step - loss: 0.3123 - accuracy: 0.8742 - val_loss: 0.3367 - val_accuracy: 0.85
Epoch 4/5
782/782 [=====] - 4s 4ms/step - loss: 0.2535 - accuracy: 0.8968 - val_loss: 0.5343 - val_accuracy: 0.80
Epoch 5/5
782/782 [=====] - 4s 4ms/step - loss: 0.1879 - accuracy: 0.9274 - val_loss: 0.3888 - val_accuracy: 0.84
<keras.callbacks.History at 0x7fa401b8f9d0>
```

We get about 84.2% accuracy on the validation set after just the first epoch, but after that the model makes no significant progress. We will do better in Chapter 16. For now the point is just to perform efficient preprocessing using `tf.data` and Keras preprocessing layers.

✓ e.

Exercise: Add an `Embedding` layer and compute the mean embedding for each review, multiplied by the square root of the number of words (see Chapter 16). This rescaled mean embedding can then be passed to the rest of your model.

To compute the mean embedding for each review, and multiply it by the square root of the number of words in that review, we will need a little function. For each sentence, this function needs to compute $M \times \sqrt{N}$, where M is the mean of all the word embeddings in the sentence (excluding padding tokens), and N is the number of words in the sentence (also excluding padding tokens). We can rewrite M as $\frac{S}{N}$, where S is the sum of all word embeddings (it does not matter whether or not we include the padding tokens in this sum, since their representation is a zero vector). So the function must return $M \times \sqrt{N} = \frac{S}{N} \times \sqrt{N} = \frac{S}{\sqrt{N} \times \sqrt{N}} \times \sqrt{N} = \frac{S}{\sqrt{N}}$.

```
def compute_mean_embedding(inputs):
    not_pad = tf.math.count_nonzero(inputs, axis=-1)
    n_words = tf.math.count_nonzero(not_pad, axis=-1, keepdims=True)
    sqrt_n_words = tf.math.sqrt(tf.cast(n_words, tf.float32))
    return tf.reduce_sum(inputs, axis=1) / sqrt_n_words

another_example = tf.constant([[[1., 2., 3.], [4., 5., 0.], [0., 0., 0.]],
                               [[6., 0., 0.], [0., 0., 0.], [0., 0., 0.]])
compute_mean_embedding(another_example)

<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[3.535534 , 4.9497476, 2.1213205],
       [6.         , 0.         , 0.         ]], dtype=float32)>
```

Let's check that this is correct. The first review contains 2 words (the last token is a zero vector, which represents the `<pad>` token). Let's compute the mean embedding for these 2 words, and multiply the result by the square root of 2:

```
tf.reduce_mean(another_example[0:1, :2], axis=1) * tf.sqrt(2.)

<tf.Tensor: shape=(1, 3), dtype=float32, numpy=array([[3.535534 , 4.9497476, 2.1213205]], dtype=float32)>
```

Looks good! Now let's check the second review, which contains just one word (we ignore the two padding tokens):

```
tf.reduce_mean(another_example[1:2, :1], axis=1) * tf.sqrt(1.)

<tf.Tensor: shape=(1, 3), dtype=float32, numpy=array([[6., 0., 0.]], dtype=float32)>
```

Perfect. Now we're ready to train our final model. It's the same as before, except we replaced TF-IDF with ordinal encoding (`output_mode="int"`) followed by an `Embedding` layer, followed by a `Lambda` layer that calls the `compute_mean_embedding` layer:

```
embedding_size = 20
tf.random.set_seed(42)

text_vectorization = tf.keras.layers.TextVectorization(
    max_tokens=max_tokens, output_mode="int")
text_vectorization.adapt(sample_reviews)

model = tf.keras.Sequential([
```

```

text_vectorization,
tf.keras.layers.Embedding(input_dim=max_tokens,
                           output_dim=embedding_size,
                           mask_zero=True), # <pad> tokens => zero vectors
tf.keras.layers.Lambda(compute_mean_embedding),
tf.keras.layers.Dense(100, activation="relu"),
tf.keras.layers.Dense(1, activation="sigmoid"),

```

✓ f.

Exercise: Train the model and see what accuracy you get. Try to optimize your pipelines to make training as fast as possible.

```

model.compile(loss="binary_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
model.fit(train_set, epochs=5, validation_data=valid_set)

```

```

Epoch 1/5
782/782 [=====] - 9s 10ms/step - loss: 0.4758 - accuracy: 0.7675 - val_loss: 0.4153 - val_accuracy: 0.8
Epoch 2/5
782/782 [=====] - 8s 9ms/step - loss: 0.3438 - accuracy: 0.8537 - val_loss: 0.3814 - val_accuracy: 0.82
Epoch 3/5
782/782 [=====] - 8s 10ms/step - loss: 0.3244 - accuracy: 0.8618 - val_loss: 0.3341 - val_accuracy: 0.8
Epoch 4/5
782/782 [=====] - 10s 11ms/step - loss: 0.3153 - accuracy: 0.8666 - val_loss: 0.3122 - val_accuracy: 0.
Epoch 5/5
782/782 [=====] - 11s 12ms/step - loss: 0.3135 - accuracy: 0.8676 - val_loss: 0.3119 - val_accuracy: 0.
<keras.callbacks.History at 0x7fa3a0bf9460>

```

The model is just marginally better using embeddings (but we will do better in Chapter 16). The pipeline looks fast enough (we optimized it earlier).

✓ g.

Exercise: Use TFDS to load the same dataset more easily: `tfds.Load("imdb_reviews")`.

```

import tensorflow_datasets as tfds

datasets = tfds.load(name="imdb_reviews")
train_set, test_set = datasets["train"], datasets["test"]

```

```

for example in train_set.take(1):
    print(example["text"])
    print(example["label"])

```

```

tf.Tensor(b"This was an absolutely terrible movie. Don't be lured in by Christopher Walken or Michael Ironside. Both are great a
tf.Tensor(0, shape=(), dtype=int64)

```

Bắt đầu lập trình hoặc tạo mã bằng trí tuệ nhân tạo (AI).